Section 11.4 - Transformer encoder for text classification

January 2, 2026

1 Transformer encoder for text classification

1.1 Transformer encoder without positional encoding

- Word embedding `Without` positional embedding
- MultiHeadAttention
- Dense projection
- Residual connection
- Layer normalization

1.1.1 Preparing the data

- The familiar IMDB dataset
- Sentiment classification
- Balanced two classes
- Use the utility function: text_dataset_from_directory()

[1]:
```
from tensorflow import keras

batch_size = 32

train_ds = keras.utils.text_dataset_from_directory(
    "aclImdb/train", batch_size=batch_size
)
val_ds = keras.utils.text_dataset_from_directory(
    "aclImdb/val", batch_size=batch_size
)
test_ds = keras.utils.text_dataset_from_directory(
    "aclImdb/test", batch_size=batch_size
)

text_only_train_ds = train_ds.map(lambda x, y: x)
```

```
Found 20000 files belonging to 2 classes.
Found 5000 files belonging to 2 classes.
Found 25000 files belonging to 2 classes.
```

1.1.2 TextVectorization

[2]:
```
from tensorflow.keras import layers

max_length = 600
max_tokens = 20000
text_vectorization = layers.TextVectorization(
    max_tokens=max_tokens,
    output_mode="int",
    output_sequence_length=max_length,
)
text_vectorization.adapt(text_only_train_ds)

int_train_ds = train_ds.map(
    lambda x, y: (text_vectorization(x), y),
    num_parallel_calls=4)
int_val_ds = val_ds.map(
    lambda x, y: (text_vectorization(x), y),
    num_parallel_calls=4)
int_test_ds = test_ds.map(
    lambda x, y: (text_vectorization(x), y),
    num_parallel_calls=4)
```

1.1.3 Transformer encoder as a subclassed Layer

- Word embedding `Without` positional embedding
- MultiHeadAttention
- Dense projection
- Residual connection
- Layer normalization

[3]:
```
import tensorflow as tf

class TransformerEncoder(layers.Layer):
    def __init__(self, embed_dim, dense_dim, num_heads, **kwargs):
        super().__init__(**kwargs)
        self.embed_dim = embed_dim
        self.dense_dim = dense_dim
        self.num_heads = num_heads
        self.attention = layers.MultiHeadAttention(
            num_heads=num_heads, key_dim=embed_dim)
        self.dense_proj = keras.Sequential(
            [layers.Dense(dense_dim, activation="relu"),
             layers.Dense(embed_dim),]
        )
        self.layernorm_1 = layers.LayerNormalization()
        self.layernorm_2 = layers.LayerNormalization()
```

```
    def call(self, inputs, mask=None):
        if mask is not None:
            mask = mask[:, tf.newaxis, :]
        attention_output = self.attention(
            inputs, inputs, attention_mask=mask)
        proj_input = self.layernorm_1(inputs + attention_output)
        proj_output = self.dense_proj(proj_input)
        return self.layernorm_2(proj_input + proj_output)

    def get_config(self):
        config = super().get_config()
        config.update({
            "embed_dim": self.embed_dim,
            "num_heads": self.num_heads,
            "dense_dim": self.dense_dim,
        })
        return config
```

1.1.4 Transformer encoder for text classification

- Word embedding
- `Transformer encoder`
- GlobalMaxPooling1D() + Dropout()
- Dense + `sigmoid` activation

[4]:
```
vocab_size = 20000
embed_dim = 256
num_heads = 2
dense_dim = 32

inputs = keras.Input(shape=(None,), dtype="int64")
x = layers.Embedding(vocab_size, embed_dim)(inputs)
x = TransformerEncoder(embed_dim, dense_dim, num_heads)(x)
x = layers.GlobalMaxPooling1D()(x)
x = layers.Dropout(0.5)(x)
outputs = layers.Dense(1, activation="sigmoid")(x)
model = keras.Model(inputs, outputs)
model.compile(optimizer="rmsprop",
              loss="binary_crossentropy",
              metrics=["accuracy"])
model.summary()
```

```
Model: "model"
_________________________________________________________________
 Layer (type)                Output Shape              Param #
=================================================================
 input_1 (InputLayer)        [(None, None)]            0
```

```
 embedding (Embedding)       (None, None, 256)         5120000

 transformer_encoder (Trans  (None, None, 256)         543776
 formerEncoder)

 global_max_pooling1d (Glob  (None, 256)               0
 alMaxPooling1D)

 dropout (Dropout)           (None, 256)               0

 dense_2 (Dense)             (None, 1)                 257

=================================================================
Total params: 5664033 (21.61 MB)
Trainable params: 5664033 (21.61 MB)
Non-trainable params: 0 (0.00 Byte)
_________________________________________________________________
```

1.1.5 Training the model

[5]:
```
# Hide the warning messages
import logging

tf.get_logger().setLevel(logging.ERROR)

import warnings
warnings.filterwarnings(
    "ignore",
    message="You are saving your model as an HDF5 file.*"
)

callbacks = [
    keras.callbacks.ModelCheckpoint("transformer_encoder.h5",
                                    save_best_only=True)
]
history = model.fit(int_train_ds, validation_data=int_val_ds, epochs=20,␣
 ↪callbacks=callbacks)
```

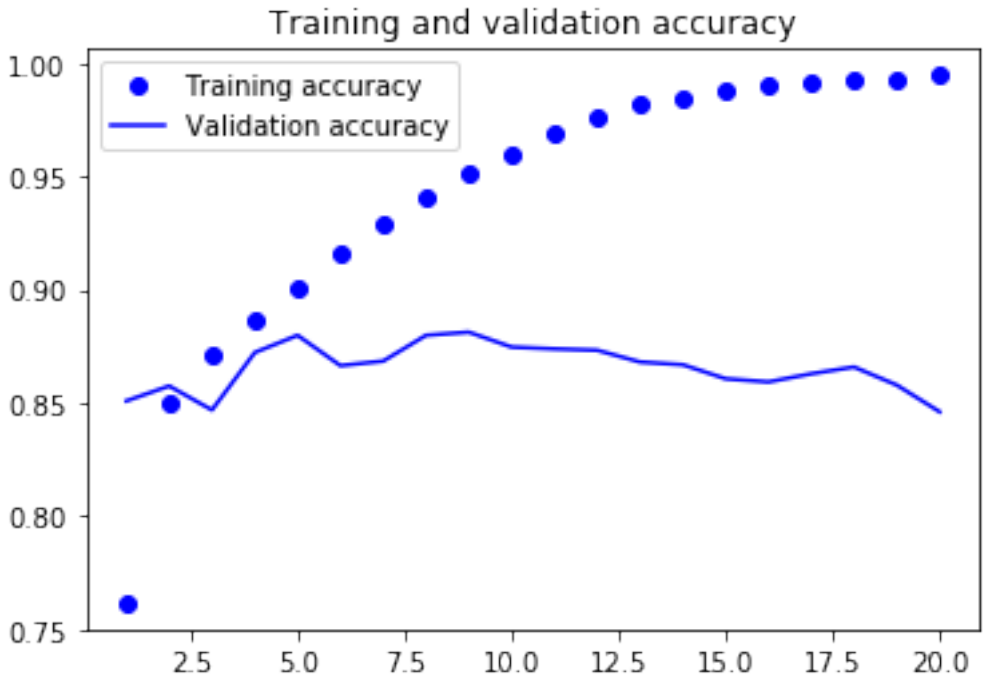
```
Epoch 1/20
625/625 [==============================] - 69s 107ms/step - loss: 0.5043 -
accuracy: 0.7615 - val_loss: 0.3609 - val_accuracy: 0.8508
Epoch 2/20
625/625 [==============================] - 45s 73ms/step - loss: 0.3450 -
accuracy: 0.8500 - val_loss: 0.3318 - val_accuracy: 0.8574
Epoch 3/20
625/625 [==============================] - 37s 60ms/step - loss: 0.3077 -
accuracy: 0.8706 - val_loss: 0.3390 - val_accuracy: 0.8468
Epoch 4/20
```

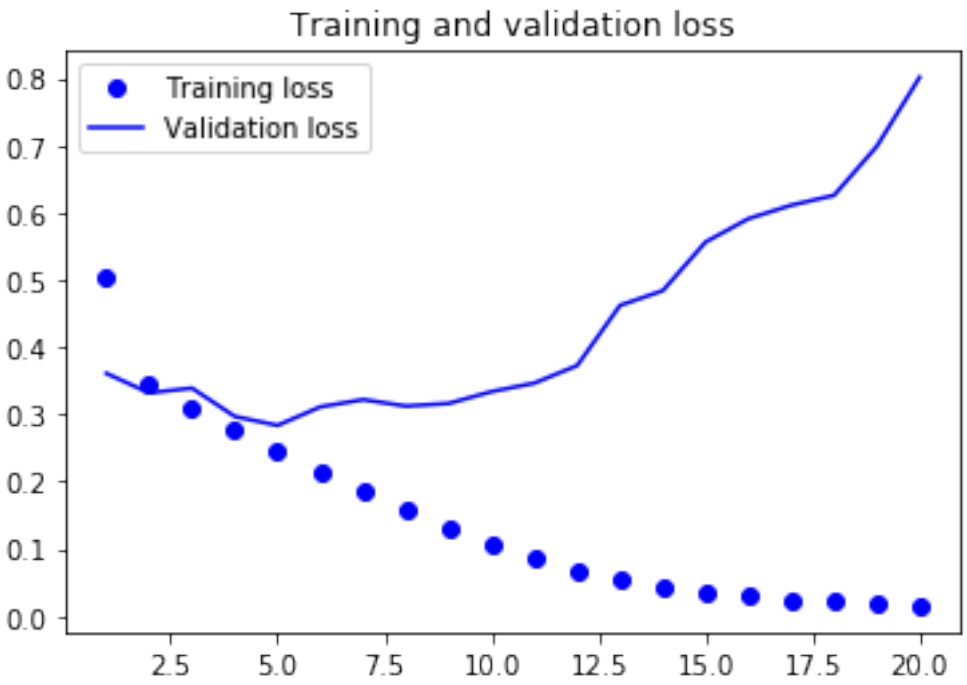
```
625/625 [==============================] - 33s 53ms/step - loss: 0.2756 -
accuracy: 0.8866 - val_loss: 0.2968 - val_accuracy: 0.8722
Epoch 5/20
625/625 [==============================] - 32s 52ms/step - loss: 0.2466 -
accuracy: 0.9008 - val_loss: 0.2835 - val_accuracy: 0.8798
Epoch 6/20
625/625 [==============================] - 33s 52ms/step - loss: 0.2136 -
accuracy: 0.9155 - val_loss: 0.3111 - val_accuracy: 0.8664
Epoch 7/20
625/625 [==============================] - 32s 51ms/step - loss: 0.1875 -
accuracy: 0.9290 - val_loss: 0.3219 - val_accuracy: 0.8684
Epoch 8/20
625/625 [==============================] - 31s 49ms/step - loss: 0.1590 -
accuracy: 0.9413 - val_loss: 0.3122 - val_accuracy: 0.8798
Epoch 9/20
625/625 [==============================] - 31s 50ms/step - loss: 0.1298 -
accuracy: 0.9514 - val_loss: 0.3162 - val_accuracy: 0.8812
Epoch 10/20
625/625 [==============================] - 31s 49ms/step - loss: 0.1063 -
accuracy: 0.9596 - val_loss: 0.3338 - val_accuracy: 0.8746
Epoch 11/20
625/625 [==============================] - 31s 50ms/step - loss: 0.0866 -
accuracy: 0.9697 - val_loss: 0.3464 - val_accuracy: 0.8738
Epoch 12/20
625/625 [==============================] - 31s 49ms/step - loss: 0.0678 -
accuracy: 0.9758 - val_loss: 0.3726 - val_accuracy: 0.8732
Epoch 13/20
625/625 [==============================] - 31s 50ms/step - loss: 0.0545 -
accuracy: 0.9819 - val_loss: 0.4619 - val_accuracy: 0.8680
Epoch 14/20
625/625 [==============================] - 31s 50ms/step - loss: 0.0441 -
accuracy: 0.9841 - val_loss: 0.4839 - val_accuracy: 0.8668
Epoch 15/20
625/625 [==============================] - 31s 49ms/step - loss: 0.0342 -
accuracy: 0.9880 - val_loss: 0.5562 - val_accuracy: 0.8606
Epoch 16/20
625/625 [==============================] - 30s 49ms/step - loss: 0.0296 -
accuracy: 0.9905 - val_loss: 0.5908 - val_accuracy: 0.8592
Epoch 17/20
625/625 [==============================] - 31s 49ms/step - loss: 0.0234 -
accuracy: 0.9912 - val_loss: 0.6109 - val_accuracy: 0.8628
Epoch 18/20
625/625 [==============================] - 30s 48ms/step - loss: 0.0212 -
accuracy: 0.9922 - val_loss: 0.6255 - val_accuracy: 0.8658
Epoch 19/20
625/625 [==============================] - 31s 50ms/step - loss: 0.0199 -
accuracy: 0.9927 - val_loss: 0.6985 - val_accuracy: 0.8578
Epoch 20/20
```

```
625/625 [==============================] - 31s 50ms/step - loss: 0.0152 -
accuracy: 0.9948 - val_loss: 0.8012 - val_accuracy: 0.8460
```

1.1.6 Plotting the results

[6]:
```
import matplotlib.pyplot as plt

acc = history.history["accuracy"]
val_acc = history.history["val_accuracy"]
loss = history.history["loss"]
val_loss = history.history["val_loss"]
epochs = range(1, len(acc) + 1)
plt.plot(epochs, acc, "bo", label="Training accuracy")
plt.plot(epochs, val_acc, "b", label="Validation accuracy")
plt.title("Training and validation accuracy")
plt.legend()
plt.figure()
plt.plot(epochs, loss, "bo", label="Training loss")
plt.plot(epochs, val_loss, "b", label="Validation loss")
plt.title("Training and validation loss")
plt.legend()
plt.show()
```





1.1.7 Evaluating on the test set

- Test accuracy: 0.876

[7]:
```
model = keras.models.load_model(
    "transformer_encoder.h5",
    custom_objects={"TransformerEncoder": TransformerEncoder})
print(f"Test acc: {model.evaluate(int_test_ds)[1]:.3f}")
```

```
782/782 [==============================] - 13s 16ms/step - loss: 0.2935 -
accuracy: 0.8757
Test acc: 0.876
```

1.2 Full Transformer encoder

- Word embedding & `positional embedding`
- MultiHeadAttention
- Dense projection
- Residual connection
- Layer normalization

1.2.1 Positional embedding as a subclassed `Layer`

- Word embedding for `all tokens`: matrix E of size (max_tokens, embed_dim)
- Positional embedding for `all indices`: matrix P of size (max_length, embed_dim)
- For a sequence: its word embedding + (a portion of) P

[8]:
```
class PositionalEmbedding(layers.Layer):
    def __init__(self, sequence_length, input_dim, output_dim, **kwargs):
        super().__init__(**kwargs)
        self.token_embeddings = layers.Embedding(
            input_dim=input_dim, output_dim=output_dim)
        self.position_embeddings = layers.Embedding(
            input_dim=sequence_length, output_dim=output_dim)
        self.sequence_length = sequence_length
        self.input_dim = input_dim
        self.output_dim = output_dim

    def call(self, inputs):
        length = tf.shape(inputs)[-1]
        positions = tf.range(start=0, limit=length, delta=1)
        embedded_tokens = self.token_embeddings(inputs)
        embedded_positions = self.position_embeddings(positions)
        return embedded_tokens + embedded_positions

    def compute_mask(self, inputs, mask=None):
        return tf.math.not_equal(inputs, 0)

    def get_config(self):
        config = super().get_config()
        config.update({
            "output_dim": self.output_dim,
            "sequence_length": self.sequence_length,
            "input_dim": self.input_dim,
        })
        return config
```

1.2.2 Model `With` positional embedding for text classification

- Word embedding
- `Positional embedding`
- Transformer encoder
- GlobalMaxPooling1D() + Dropout()
- Dense + `sigmoid` activation

[9]:
```
vocab_size = 20000
sequence_length = 600
embed_dim = 256
num_heads = 2
dense_dim = 32

inputs = keras.Input(shape=(None,), dtype="int64")
x = PositionalEmbedding(sequence_length, vocab_size, embed_dim)(inputs)
x = TransformerEncoder(embed_dim, dense_dim, num_heads)(x)
```

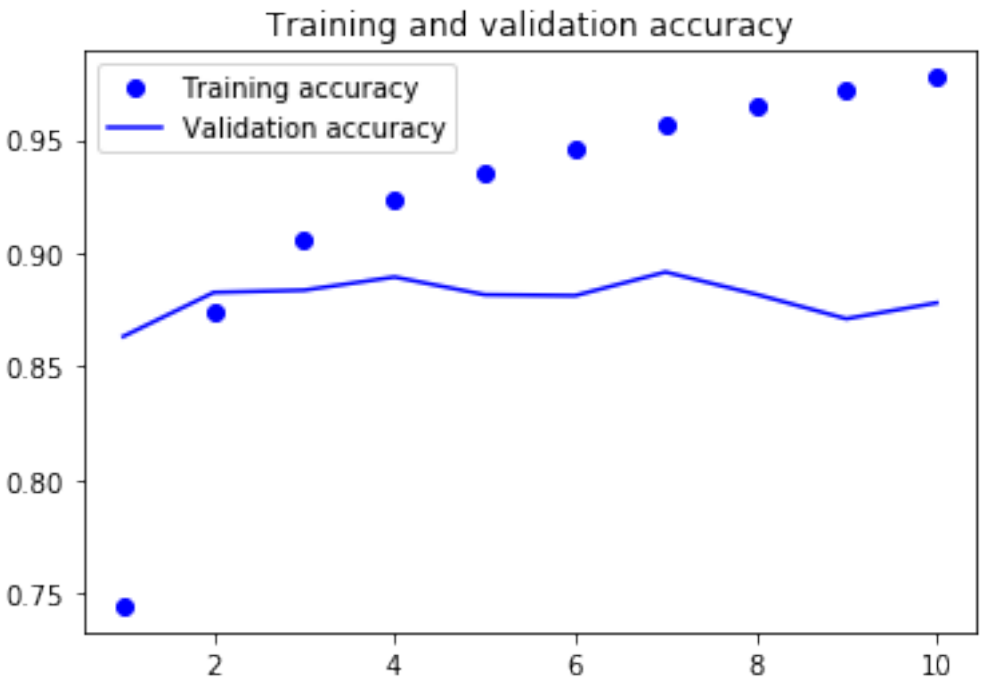
```
x = layers.GlobalMaxPooling1D()(x)
x = layers.Dropout(0.5)(x)
outputs = layers.Dense(1, activation="sigmoid")(x)
model_full = keras.Model(inputs, outputs)
model_full.compile(optimizer="rmsprop",
              loss="binary_crossentropy",
              metrics=["accuracy"])
model_full.summary()

callbacks = [
    keras.callbacks.ModelCheckpoint("full_transformer_encoder.h5",
                                    save_best_only=True)
]
history = model_full.fit(int_train_ds, validation_data=int_val_ds, epochs=10,␣
 ↪callbacks=callbacks)
```

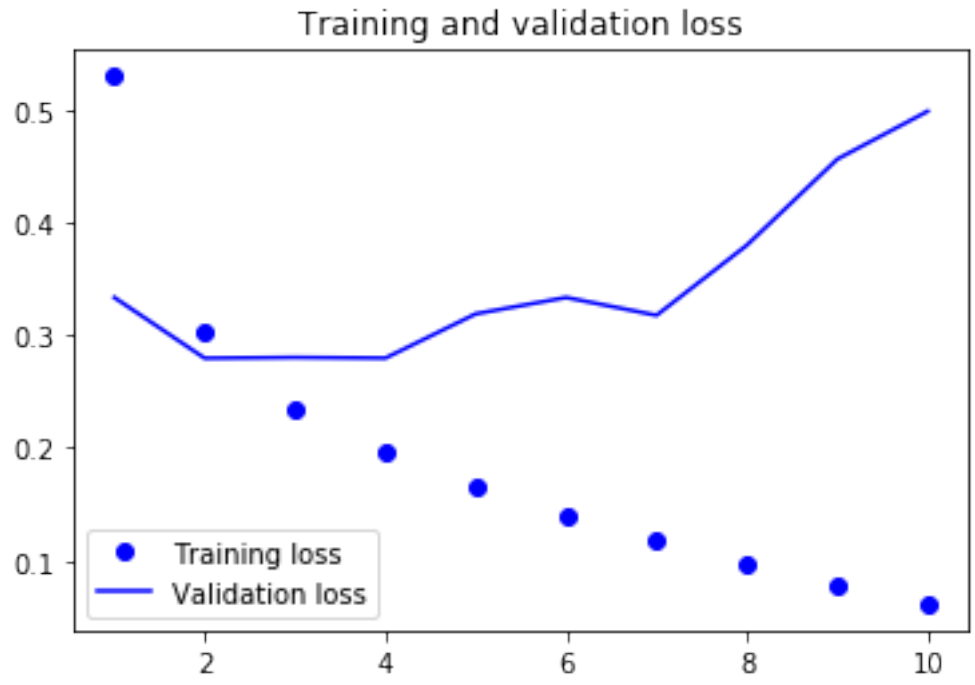
```
Model: "model_1"
_________________________________________________________________
 Layer (type)                Output Shape              Param #
=================================================================
 input_2 (InputLayer)        [(None, None)]            0

 positional_embedding (Posi  (None, None, 256)         5273600
 tionalEmbedding)

 transformer_encoder_1 (Tra  (None, None, 256)         543776
 nsformerEncoder)

 global_max_pooling1d_1 (Gl  (None, 256)               0
 obalMaxPooling1D)

 dropout_1 (Dropout)         (None, 256)               0

 dense_7 (Dense)             (None, 1)                 257

=================================================================
Total params: 5817633 (22.19 MB)
Trainable params: 5817633 (22.19 MB)
Non-trainable params: 0 (0.00 Byte)
_________________________________________________________________
Epoch 1/10
625/625 [==============================] - 59s 91ms/step - loss: 0.5294 -
accuracy: 0.7444 - val_loss: 0.3332 - val_accuracy: 0.8632
Epoch 2/10
625/625 [==============================] - 42s 68ms/step - loss: 0.3026 -
accuracy: 0.8745 - val_loss: 0.2794 - val_accuracy: 0.8826
Epoch 3/10
625/625 [==============================] - 38s 61ms/step - loss: 0.2342 -
```

```
accuracy: 0.9058 - val_loss: 0.2801 - val_accuracy: 0.8836
Epoch 4/10
625/625 [==============================] - 34s 54ms/step - loss: 0.1951 -
accuracy: 0.9236 - val_loss: 0.2795 - val_accuracy: 0.8894
Epoch 5/10
625/625 [==============================] - 33s 53ms/step - loss: 0.1666 -
accuracy: 0.9356 - val_loss: 0.3187 - val_accuracy: 0.8816
Epoch 6/10
625/625 [==============================] - 33s 53ms/step - loss: 0.1405 -
accuracy: 0.9464 - val_loss: 0.3332 - val_accuracy: 0.8812
Epoch 7/10
625/625 [==============================] - 34s 54ms/step - loss: 0.1173 -
accuracy: 0.9571 - val_loss: 0.3174 - val_accuracy: 0.8916
Epoch 8/10
625/625 [==============================] - 33s 52ms/step - loss: 0.0966 -
accuracy: 0.9651 - val_loss: 0.3796 - val_accuracy: 0.8818
Epoch 9/10
625/625 [==============================] - 33s 53ms/step - loss: 0.0784 -
accuracy: 0.9717 - val_loss: 0.4558 - val_accuracy: 0.8710
Epoch 10/10
625/625 [==============================] - 33s 53ms/step - loss: 0.0616 -
accuracy: 0.9775 - val_loss: 0.4983 - val_accuracy: 0.8780
```

1.2.3 Plotting the results

[10]:
```
acc = history.history["accuracy"]
val_acc = history.history["val_accuracy"]
loss = history.history["loss"]
val_loss = history.history["val_loss"]
epochs = range(1, len(acc) + 1)
plt.plot(epochs, acc, "bo", label="Training accuracy")
plt.plot(epochs, val_acc, "b", label="Validation accuracy")
plt.title("Training and validation accuracy")
plt.legend()
plt.figure()
plt.plot(epochs, loss, "bo", label="Training loss")
plt.plot(epochs, val_loss, "b", label="Validation loss")
plt.title("Training and validation loss")
plt.legend()
plt.show()
```


Training and validation accuracy
Training accuracy
Validation accuracy
0.95
0.90
0.85
0.80
0.75
2
4
6
8
10


Training and validation loss
0.5
0.4
0.3
0.2
0.1
Training loss
Validation loss
2
4
6
8
10

1.2.4 Evaluating on the test set

- Test accuracy: 0.878 compared to 0.876 of without positional embedding

[11]:
```
model = keras.models.load_model(
    "full_transformer_encoder.h5",
    custom_objects={"TransformerEncoder": TransformerEncoder,
                    "PositionalEmbedding": PositionalEmbedding})
print(f"Test acc: {model.evaluate(int_test_ds)[1]:.3f}")
```

```
782/782 [==============================] - 16s 20ms/step - loss: 0.2894 -
accuracy: 0.8776
Test acc: 0.878
```